

Schema de Prompt pour nous guider dans l'exercice :

Pour commencer, nous avons fait un prompt qui sert à encadrer l'exercice et aussi le LLM :

```
salut mec , je dois faire un exo en groupe je t'ai mis le contexte le but :  
nous guider dans cet exo sans donner les réponses. la visée educative  
est annhilie si tu me dit tout  
  
Ok — je vais te guider en mode "coach" : questions + étapes, checklists, et mini-exemples neutres (jamais la  
solution complète). On va d'abord cadrer l'API, la structure des données, puis l'UI et le workflow  
Git/README/LLM.  
  
Réflexion durant 59s >  
  
Parfait, je te guide en mode coach : je te donne une méthode, des checklists, des questions à vous poser, et  
des exemples neutres (avec placeholders), mais pas la solution.
```

```
0) Le "contrat" de l'exo (à garder en tête)
Votre job = comprendre les données + faire une UI claire + utiliser un framework moderne + documenter  
(README + LLM.md).

👉 Donc on vise :

• une app qui consomme l'API proprement (loading / erreurs / états vides),
• une UI avec au moins 2-3 fonctionnalités visibles,
• un dépôt Git propre (commits, branches),
• une doc claire.
```

On observe que le LLM se donne en consigne de ne pas révéler la totalité du code afin de nous permettre de rechercher nos propres lignes et donc ne pas détruire la visée éducative du projet.

```
pour commencer explique ce qu'est react

React, c'est une bibliothèque JavaScript (pas un framework "tout-en-un") faite pour construire des  
interfaces utilisateur (UI), surtout des applis web où l'écran change souvent (recherche, filtres, listes,  
formulaires...).

1) L'idée centrale : l'UI = une fonction de l'état

Au lieu de "modifier le DOM à la main" (genre document.querySelector(...).innerHTML = ...), React te  
pousse à penser :

| À un instant T, si j'ai telles données (état), alors l'écran doit ressembler à ça.

Quand les données changent, React re-rend automatiquement ce qu'il faut.
```

Afin de partir sur de bonnes bases, n'ayant jamais utilisé React.js nous avons demandé une explication

parle moi de la structure des projets react . qu'est ce qui differe d'un projet html css js classique

Dans un projet HTML/CSS/JS "classique", tu as souvent :

- des fichiers HTML (une ou plusieurs pages),
- du JS qui "branche" des écouteurs et **manipule le DOM** (`querySelector`, `innerHTML`, etc.),
- des scripts chargés dans la page,
- et tu gères toi-même quand / comment l'UI doit se mettre à jour.

Dans un projet **React**, tu changes complètement de logique : tu construis une **appli** composée de **composants**, et l'UI se met à jour automatiquement quand l'état change.

simple de react.

2) Les composants : des "briques" d'interface

En React, tu construis ton appli avec des **composants**.

- Un composant = une fonction (souvent) qui **retourne du JSX** (une syntaxe qui ressemble à du HTML).
- Tu peux réutiliser les composants comme des Legos.

Exemples de composants typiques :

- `App` (racine)
- `SearchBar`
- `ItemCard`
- `ItemList`
- `DetailPage`

3) JSX : du HTML... mais dans du JavaScript

JSX te permet d'écrire une structure "style HTML" directement dans ton code JS.

Point important : c'est pas du vrai HTML, c'est transformé en JavaScript.

4) Props vs State (super important)

C'est LA base pour comprendre comment les données circulent.

Props (propriétés) :

- données **passées d'un parent vers un enfant**
- en général, c'est "lecture seule" côté enfant

State (état) :

- données **internes** à un composant, qui peuvent changer
- quand le state change → React met à jour l'UI

📌 Règle simple :

- *Le parent possède les données, l'enfant affiche via props.*

5) Les hooks (dans le React moderne)

Les hooks sont des fonctions React qui ajoutent des "pouvoirs" à tes composants.

- `useState` : stocker/modifier un état (ex: texte de recherche, résultats, loading)
- `useEffect` : faire un effet "après rendu" (ex: appeler une API quand la page s'affiche ou quand un param change)

Pour votre projet API, vous allez quasiment forcément utiliser ces deux-là (ou un outil autour, genre React Query, mais c'est un autre niveau).

Toujours dans l'optique de progresser dans la visualisation de react j'ai demandé a chatGPT de m'illustrer les différents composant de celui-ci.

illustre moi tout ça que je vois a quoi ça ressemble

Ok, je t'illustre ça avec des **mini-bouts de React** (pas un projet complet), juste pour que tu voies à **quoi ça ressemble** : composants, JSX, props, state, useEffect, et les états loading/error/empty.

1) Un composant React ultra simple (JSX)

Un composant = une fonction qui **retourne du JSX** (un "HTML-like" dans du JS).

```
jsx
function Hello() {
  return <h1>Salut 🍌 </h1>;
}
```

 Copier le code

👉 Ici, React affichera un `<h1>` dans la page.

Voulant faire preuve d'originalité, nous avons choisi une API publique, Rick and Morty, qui colle parfaitement aux prérogatives éducatives du Brief.

est que tu peux me recommander une api gratuite pour le projet

Je vais te proposer quelques APIs publiques gratuites (avec ce qu'on peut en faire côté UI) et te donner des critères pour choisir selon les fonctionnalités voulues (liste/détail/recherche/pagination), sans vous imposer une solution.

Réflexion durant 34s >

Oui — voilà des **APIs gratuites** (ou avec un plan gratuit) qui marchent super bien pour un projet React "liste / détail / recherche / filtres", avec une UI sympa. Je te mets aussi **comment choisir** sans vous imposer une réponse unique.

Mes recommandations (faciles et pédagogiques)

1) Rick and Morty API (le meilleur "tout terrain" pour une démo UI)

Pourquoi c'est top :

- data déjà prête pour **cartes** (noms + images),
- endpoints clairs (characters/locations/episodes),
- facile de faire liste → **détail, search, pagination**. The Rick and M... +1

Idées de features :

- liste paginée + recherche par nom,
- page détail (avec infos + liens liés à l'univers).



pour faire tourner une api en javascript faut nodejs

 Partager

Ça dépend de ce que tu veux dire par "faire tourner une API" 🍌

1) Consommer une API en JavaScript (côté navigateur / React)

Non, tu n'as pas besoin de Node.js.

Si ton app React appelle une **API publique** (Rick & Morty, PokéAPI, etc.), ton navigateur peut faire les requêtes avec `fetch`.

Node.js sert surtout au **build/dev server** de React (Vite/Next), pas parce que l'API a besoin de Node.

✅ Cas typique :

- React tourne en dev via Vite (qui utilise Node)
- l'API est externe (internet)
- le navigateur appelle l'API directement

👉 Donc : **pas besoin de Node pour l'API**, mais souvent Node est installé pour lancer le projet React.

Comment créer un projet react – le point de départ de notre projet.

Étape 4 — Créer un projet React (méthode simple)

Si vous utilisez Vite (souvent le plus simple) :

```
bash
```

[Copier le code](#)

```
npm create vite@latest
```

Puis tu suis les choix :

- Project name : `rm-react` (ou autre)
- Framework : `React`
- Variant : `JavaScript`

Ensuite :

```
bash
```

[Copier le code](#)

```
cd rm-react  
npm install  
npm run dev
```

🚀 Là tu as un vrai projet React qui tourne.

Des demandes suivantes les étapes du Brief :

retour à l'exo : nos données fetch ds la console mtn il nous faut les afficher sur la page

Je vais te guider pour passer de "console.log(data)" à "afficher à l'écran" en React : créer des states (data/loading/error), déclencher le fetch au bon moment, mapper `results` en composants, et gérer les états (loading/empty/error) sans te donner le projet complet.

Ok, vous avez validé l'étape 2 (données en console). Maintenant l'étape suivante, c'est juste : **mettre ces données dans un state React puis les afficher dans le JSX.**

Je te donne la démarche "standard" (sans faire tout le projet).

du coup il nous faut afficher les info détaillés

Ok — maintenant que vous avez la liste + filtres, "afficher les infos détaillées" veut dire : **quand je clique sur un item**, j'accède à un écran (ou une zone) qui montre **plus de champs**.

Il y a 2 façons propres (choisissez 1 selon votre exo) :

Option 1 (simple) : détail "sur la même page" après un clic

Idée

- Tu affiches la liste
- Au clic sur un item, tu stockes son `id` (ou l'objet) dans un state
- Tu affiches un bloc "Détail" à droite/en dessous
- Et tu peux **fetch le détail** avec l'`id` (recommandé si vous voulez des infos supplémentaires)

States typiques

- `selectedId`
- `détail` (objet)
- `détailLoading`, `détailError`

Workflow

1. `onClick={() => setSelectedId(it.id)}`
2. `useEffect` dépendant de `selectedId` → fetch `./../{id}`
3. Afficher les champs du `détail` ↓

✔ **Avantage** : pas besoin de routing, très rapide à faire

guide nous pour faire un selecteur de page

Ok — pour faire un **sélecteur de page** (pagination) propre avec vos filtres + fetch, l'idée est simple :

- vous avez un state `page`
- vous récupérez `totalPages` depuis la réponse API (souvent dans `info.pages`)
- vous construisez l'URL avec `page` + vos filtres
- vous refetch quand `page` change

comment verifie que le resultat item existe dans le code avant de faire courir item sans donner de code

Partager

Tu veux éviter le cas où tu essaies d'afficher `item.xxx` alors que `item` n'existe pas encore (au début, pendant le fetch, ou si l'API renvoie rien). Sans code, voilà les méthodes "mentales" et les techniques React à utiliser.

1) La règle de base

Avant d'accéder à un champ (`item.name`, `item.image`, etc), assure-toi que :

- `item` n'est pas `null` / `undefined`
- et si tu accèdes à une liste interne, que cette liste existe aussi

En gros : ne lis pas un objet avant qu'il soit chargé.

2) Les 3 façons classiques de vérifier "item existe"

A) Garde conditionnel ("guard clause")

Tu fais un test en haut du rendu :

- si item n'existe pas → tu affiches un message / rien / un loader
- sinon → tu affiches le détail

Idée : "si pas prêt, je ne rends pas le bloc détail".